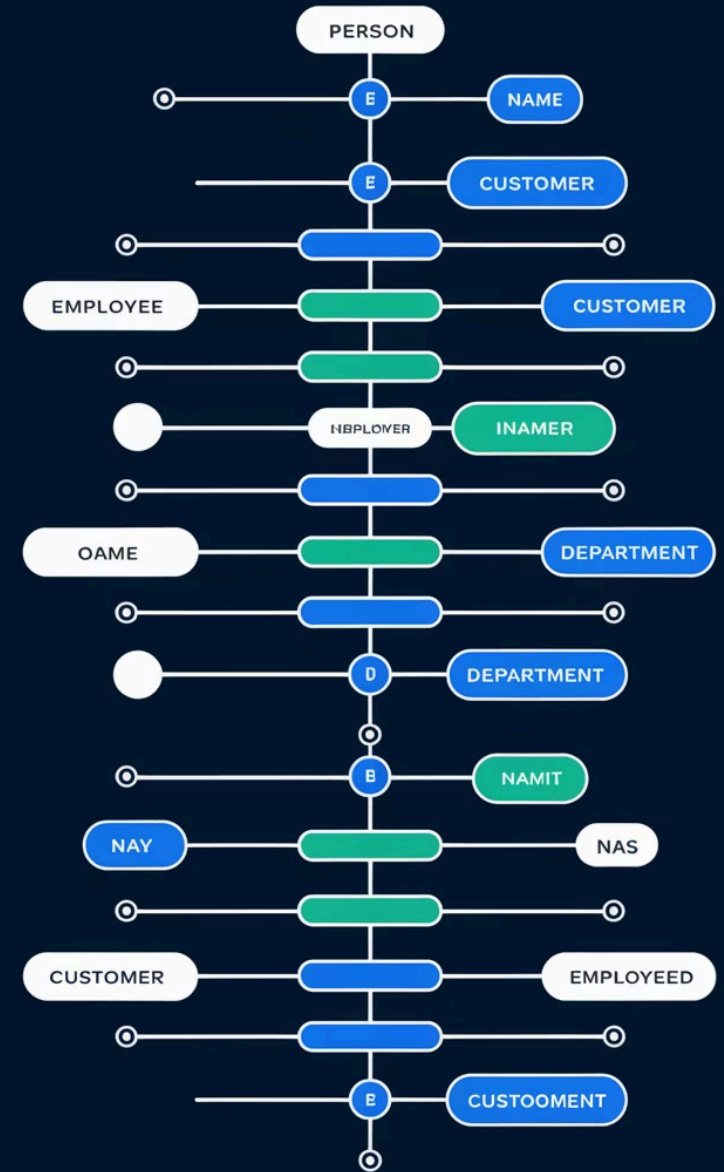


Diagrama de Classes em UML

Nesta aula, exploraremos o Diagrama de Classes, uma das ferramentas mais importantes do UML para modelar sistemas orientados a objetos.

Abordaremos conceitos, notações e exemplos práticos para facilitar sua compreensão e aplicação.

G por Giovanni Amorim



O que é um Diagrama de Classes?

Definição

Representa a estrutura estática de um sistema, mostrando classes, atributos, métodos e relacionamentos.

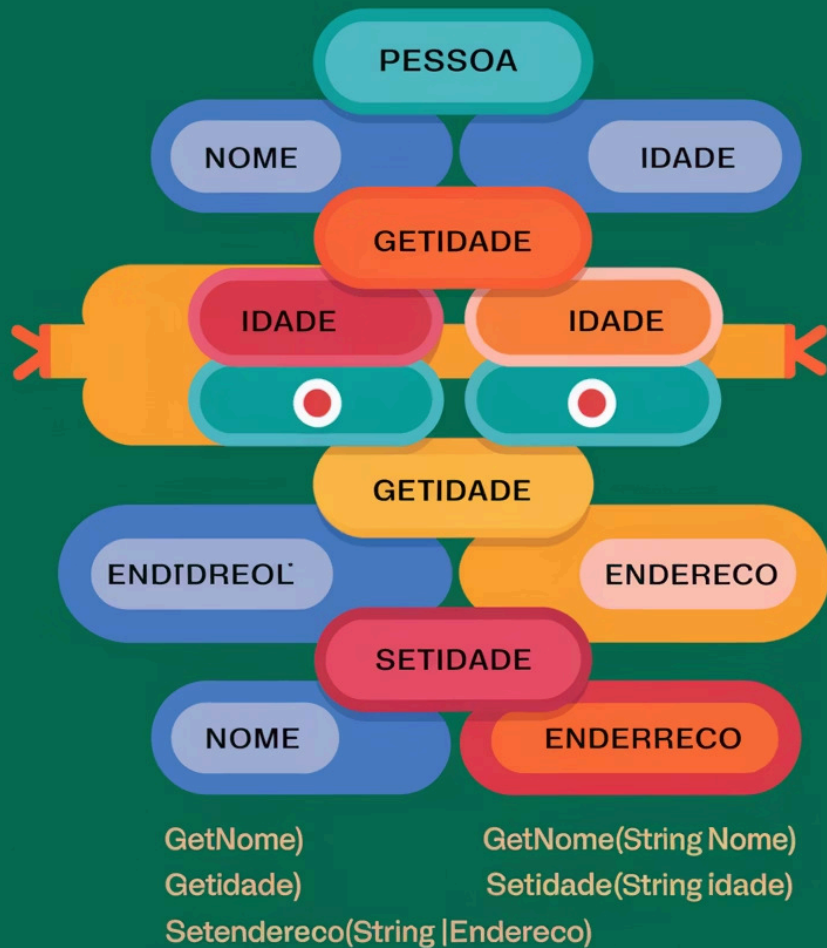
Utilidade

Facilita a visualização da arquitetura do sistema. Melhora a comunicação entre desenvolvedores e stakeholders.

Contexto no UML

É um dos diagramas estruturais mais importantes do UML.





Elementos Básicos de um Diagrama de Classes



Classe

Retângulo dividido
em três partes: nome
da classe (superior),
atributos
(intermediária) e
métodos (inferior).



Relacionamentos

Conexões entre classes que indicam interações ou dependências.



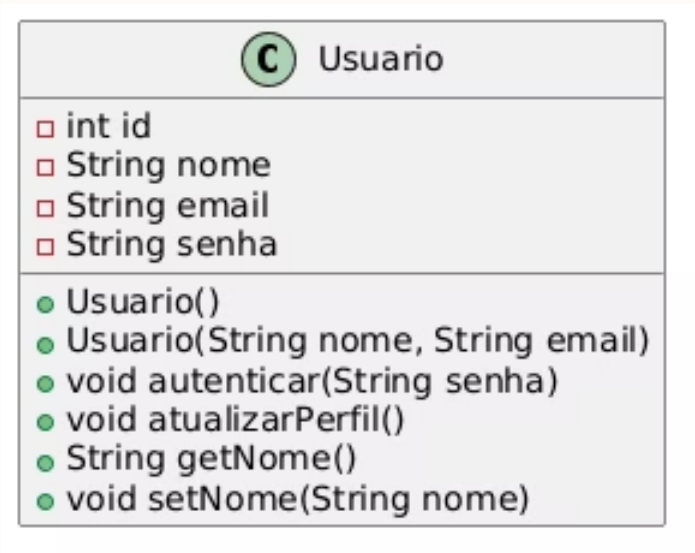
Visibilidade

Indica o nível de acesso: + (público), - (privado), # (protegido).

Relação entre Diagrama de Classes e Código Java

Um diagrama de classes UML se traduz diretamente em código orientado a objetos. Veja como os elementos do diagrama são implementados em Java:

Diagrama de Classes (UML)



No diagrama UML, a classe **Usuario** é representada com:

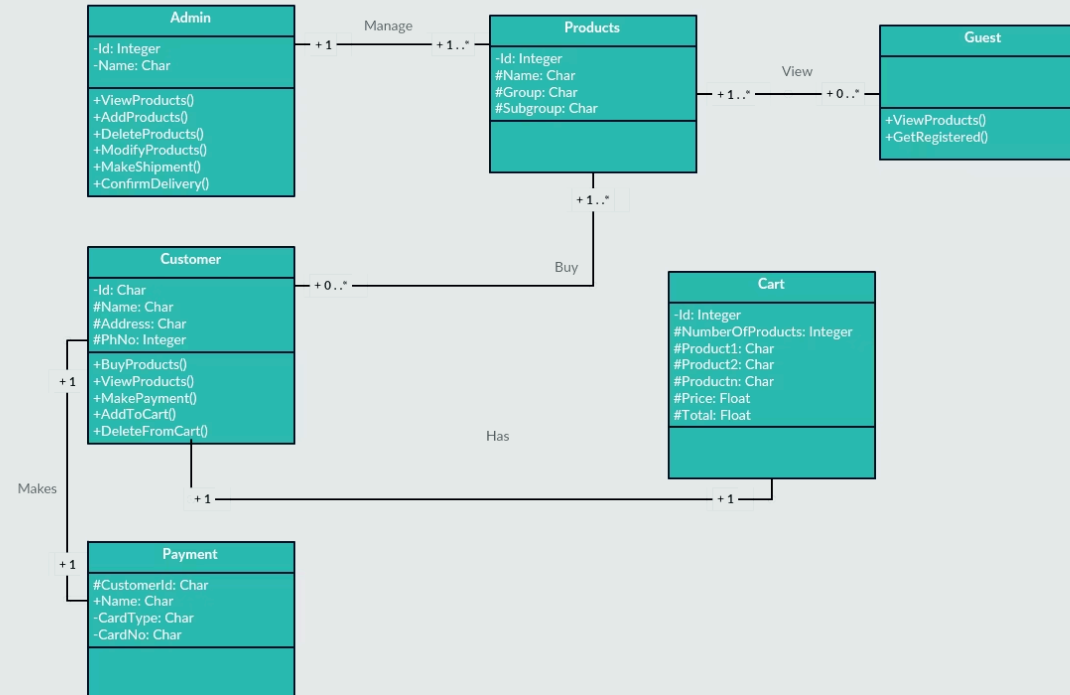
- Nome da classe no topo
- Atributos no meio (com visibilidade)
- Métodos na parte inferior

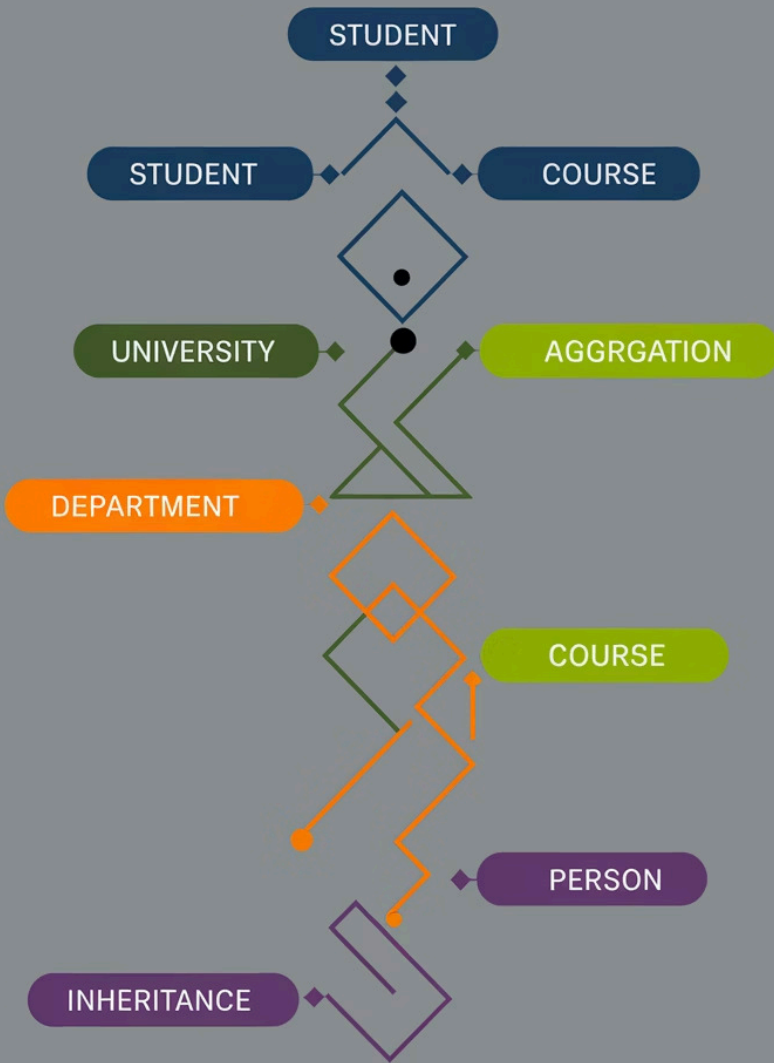
Implementação em Java

```
public class Usuario {  
    // Atributos  
    private int id;  
    private String nome;  
    private String email;  
    private String senha;  
  
    // Métodos construtores  
    public Usuario() { }  
  
    public Usuario(String nome, String email) {  
        this.nome = nome;  
        this.email = email;  
    }  
  
    // Métodos  
    public void autenticar(String senha) {  
        // implementação  
    }  
  
    public void atualizarPerfil() {  
        // implementação  
    }  
  
    // Getters e Setters  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Observe como cada elemento do diagrama UML tem uma correspondência direta no código Java: visibilidade (+/- → public/private), atributos (variáveis de instância) e métodos (funções da classe).

Online Shopping System





Tipos de Relacionamentos

Associação

Representa uma conexão simples entre duas classes. Indica que as classes se comunicam entre si. Exemplo: Professor ensina Aluno (uma linha contínua entre as classes).

Agregação

Relação "todo-parte", onde a parte pode existir independentemente do todo. Representada por um losango vazio na extremidade do todo. Exemplo: Universidade tem Departamentos (os departamentos podem existir sem a universidade).

Composição

Relação "todo-parte" mais forte, onde a parte não pode existir sem o todo. Representada por um losango preenchido. Exemplo: Casa tem Quartos (os quartos não existem sem a casa).

Herança

Representa uma relação "é-um", onde uma classe filha herda características da classe pai. Representada por uma seta com triângulo vazio. Exemplo: Veículo é generalizado por Carro e Moto.

Dependência

Indica que uma classe usa ou depende de outra, mas sem uma associação permanente. Representada por uma linha tracejada com uma seta. Exemplo: Uma classe Impressora depende de uma classe Documento para imprimir.

Realização

Ocorre quando uma classe implementa os comportamentos definidos por uma interface. Representada por uma linha tracejada com um triângulo vazio. Exemplo: A classe ContaCorrente realiza a interface Tributável.

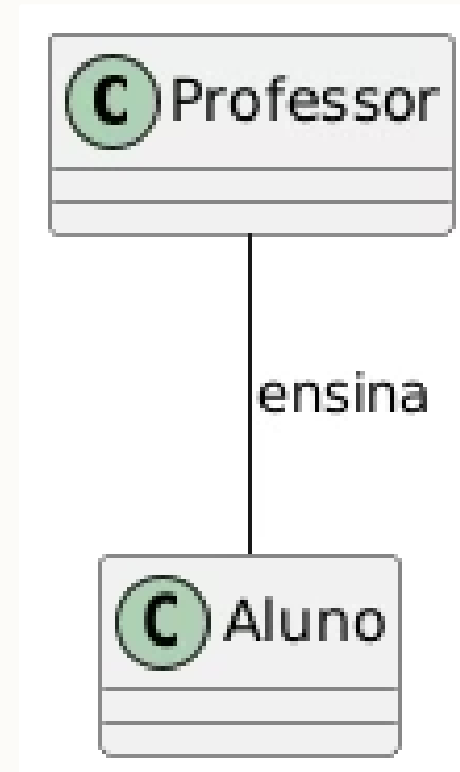
Associação

Representa uma conexão genérica entre duas classes em diagramas UML. É um dos tipos de relacionamentos mais comuns e fundamentais.

Características da Associação

- Representa uma conexão simples entre duas classes
- Indica que as classes se comunicam entre si
- É representada por uma linha contínua entre as classes
- Pode ter um nome para descrever a natureza da relação
- Pode incluir direção para indicar quem inicia a comunicação

Exemplo: **Professor ensina Aluno** - Uma linha contínua conecta as duas classes, indicando que existe uma relação entre elas. A palavra "ensina" descreve a natureza desta associação.



Agregação

Representa uma relação "todo-parte", onde a parte pode existir independentemente do todo. É um tipo específico de associação que indica uma relação mais forte, mas sem dependência de existência.

Descrição

Representa uma relação "todo-parte", onde a parte pode existir independentemente do todo.

Uso

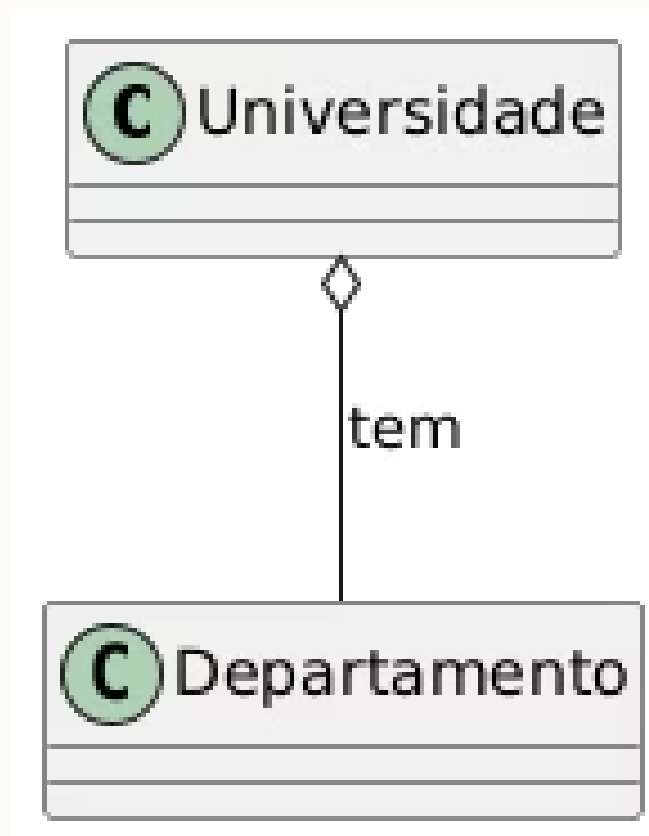
Indica que uma classe é composta por outra, mas a parte pode existir sem o todo.

Notação

Uma linha com um losango vazio na extremidade do "todo".

Exemplo Prático

Universidade tem Departamentos - os departamentos podem existir sem a universidade. O losango vazio fica do lado da Universidade, indicando que ela é o "todo" nesta relação.



Composição

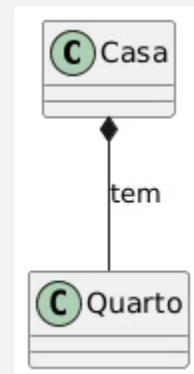
Representa uma relação "todo-parte", onde a parte não pode existir sem o todo.

1 Características da Composição

- Indica uma dependência forte entre as classes
- A parte só existe enquanto o todo existir
- Se o todo for destruído, as partes também são
- É representada por uma linha com um losango preenchido na extremidade do "todo"

2 Exemplo Prático

Casa tem Quartos - os quartos não podem existir sem a casa. O losango preenchido fica do lado da Casa, indicando que ela é o "todo" nesta relação, e os quartos são partes que dependem dela para existir.



Herança (Generalização/Especialização)

Representa uma relação "é-um", onde uma classe é uma especialização de outra.

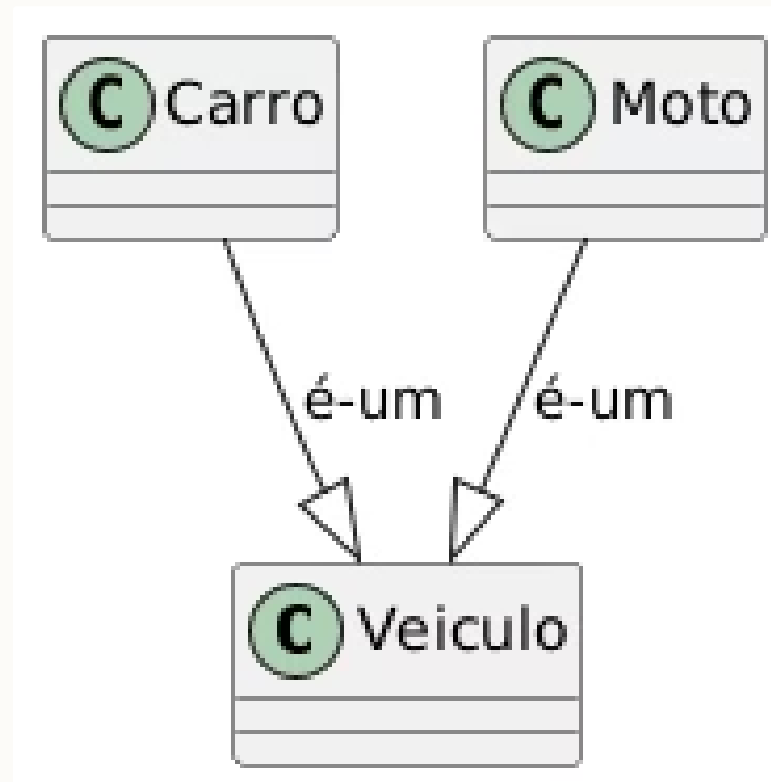
Características da Herança:

Descrição: Representa uma relação "é-um", onde uma subclasse herda características de uma superclasse, sendo uma especialização dela.

Uso: Indica que uma classe herda atributos e métodos de outra, permitindo reuso de código e modelagem de hierarquias.

Notação: Uma seta com triângulo vazio apontando para a classe pai (superclasse).

Exemplo Prático: "Veículo" é a superclasse, enquanto "Carro" e "Moto" são subclasses que herdam de "Veículo". A seta com triângulo vazio aponta das subclasses para a superclasse.



Dependência

Descrição

Representa uma relação onde uma classe depende de outra para realizar alguma funcionalidade.

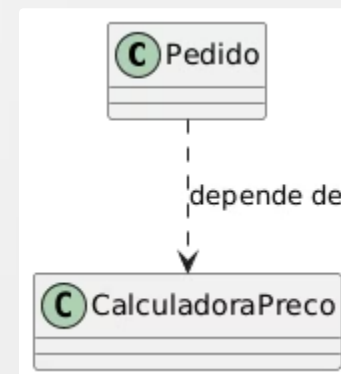
Uso

Indica que uma classe usa temporariamente outra, mas não possui uma conexão forte.

Notação

Uma seta tracejada apontando para a classe de dependência.

Exemplo Prático: Uma classe "Pedido" que depende da classe "CalculadoraPreco". O Pedido utiliza a CalculadoraPreco para calcular seu valor final, mas não mantém uma relação permanente com ela. A seta tracejada aponta do "Pedido" para a "CalculadoraPreco".



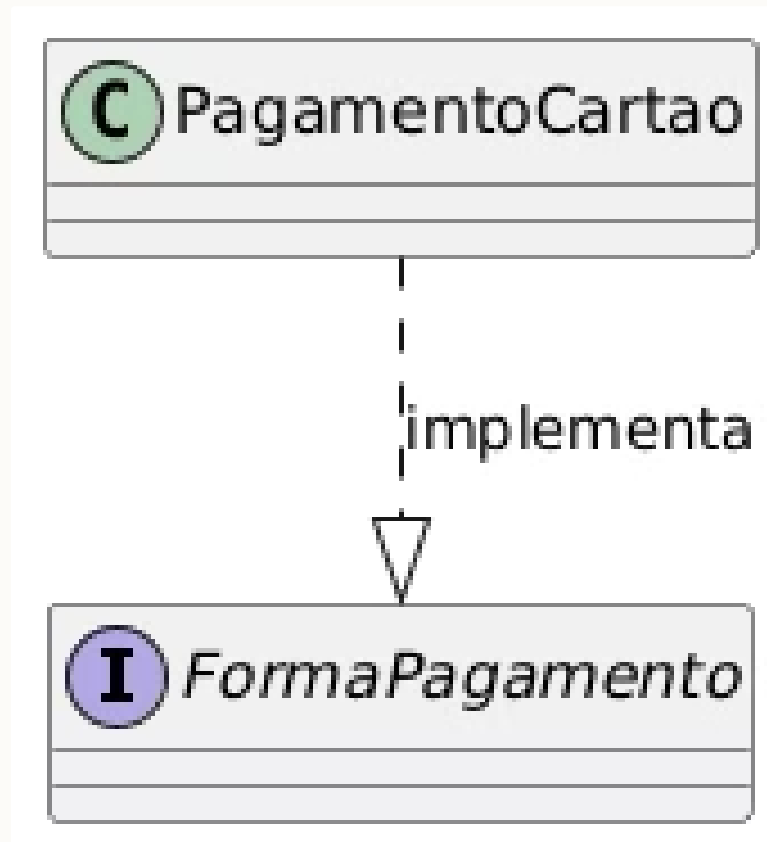
Realização (Interface)

Representa uma relação onde uma classe implementa os métodos definidos por uma interface.

Características

- **Uso:** Indica que uma classe fornece a implementação concreta dos métodos declarados em uma interface.
- **Notação:** Uma seta com triângulo vazio tracejado apontando da classe para a interface.

Exemplo Prático: Uma interface "FormaPagamento" e uma classe "PagamentoCartao" que implementa esta interface. A seta tracejada com triângulo vazio aponta da "PagamentoCartao" para a interface "FormaPagamento", indicando que "PagamentoCartao" realiza (implementa) "FormaPagamento".



Resumo dos Relacionamentos

Tipo	Descrição	Notação
Associação	Conexão genérica entre classes	Linha simples
Agregação	Todo-parte (parte pode existir sozinha)	Losango vazio
Composição	Todo-parte (parte não existe sozinha)	Losango preenchido
Herança	É-um	Seta com triângulo vazio
Dependência	Classe depende	Seta tracejada

Estes relacionamentos são fundamentais para modelar as interações entre classes em um diagrama UML, permitindo representar diferentes níveis de acoplamento e dependência.

Exemplo Completo com Todos os Relacionamentos

Veja abaixo um exemplo que combina todos os tipos de relacionamentos em um único diagrama:

Agregação

Universidade o-- Departamento: "tem" (o departamento pode existir sem a universidade)

Composição

Departamento *-- Professor: "tem" (professor não existe sem o departamento)

Associação

Professor -- Aluno: "ensina" (relacionamento simples)

Dependência

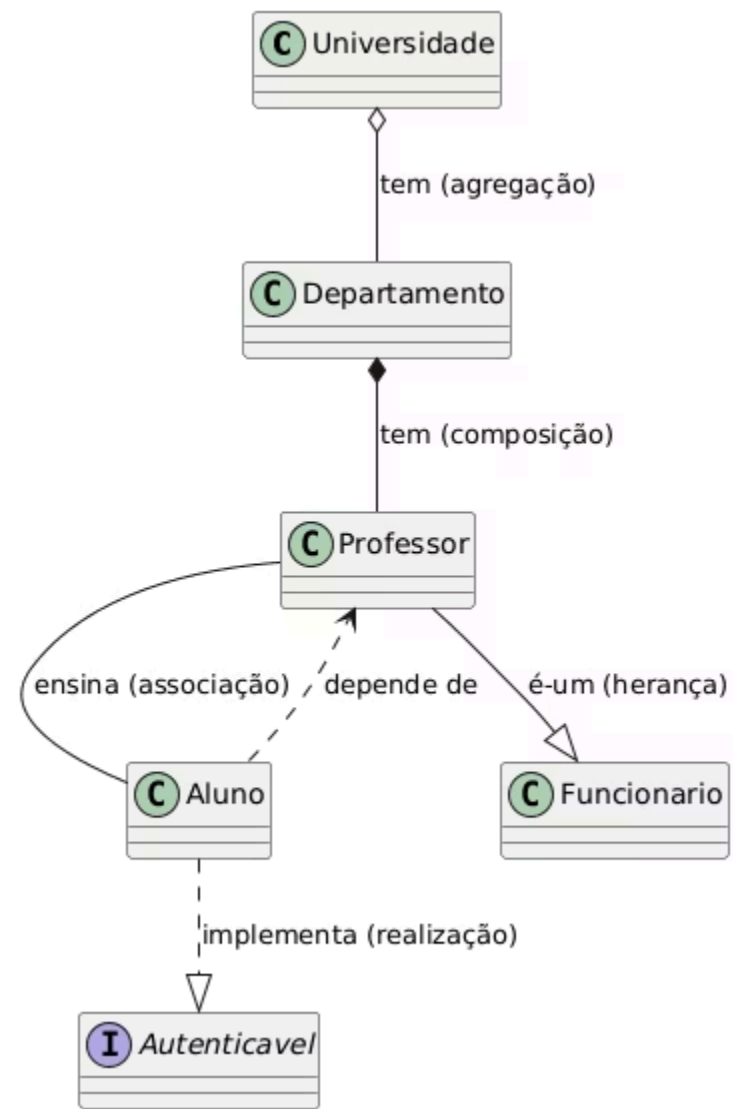
Aluno ..> Professor: "depende de" (usa temporariamente)

Realização

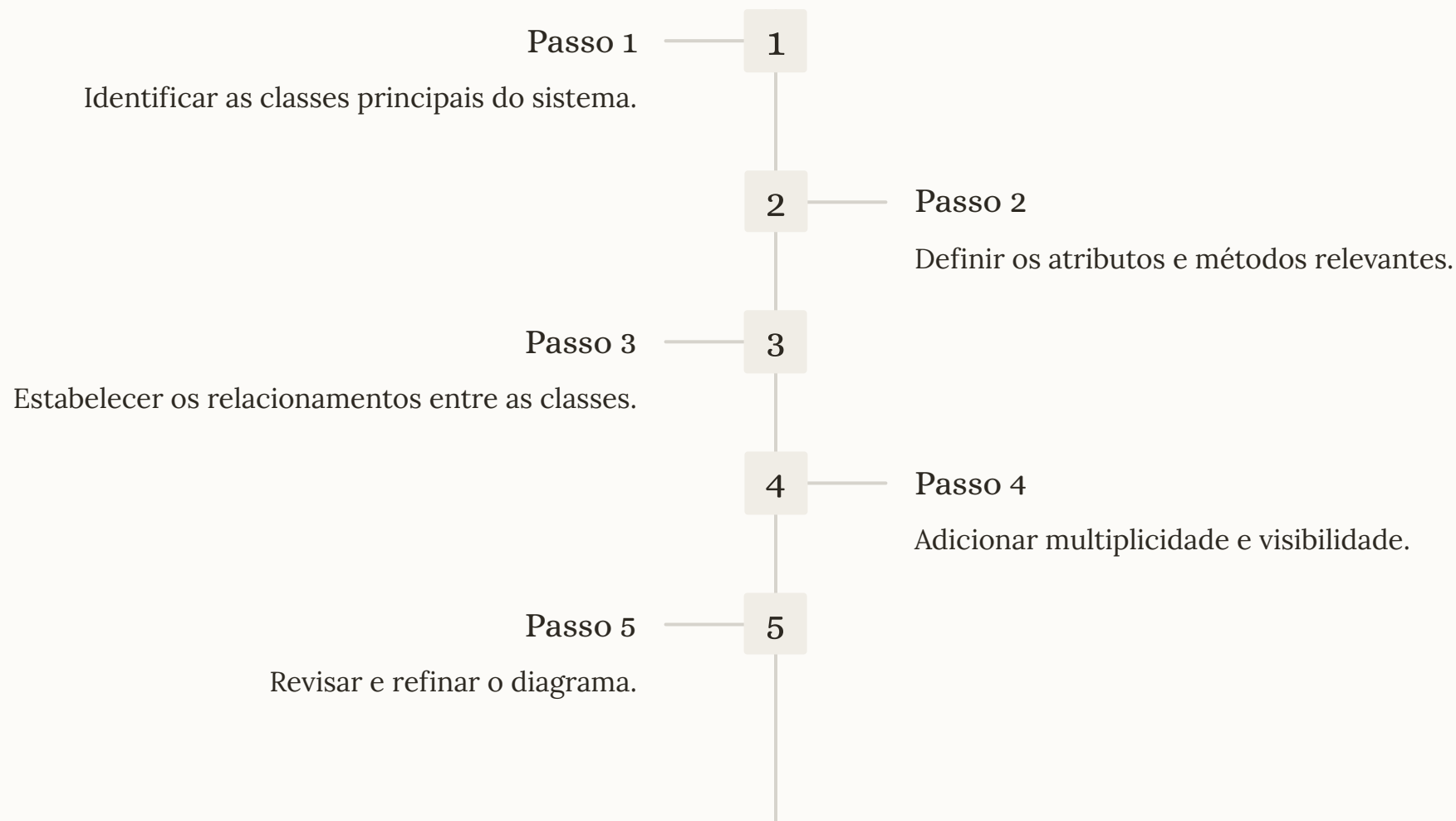
Aluno ..|> Autenticavel: "implementa" (interface)

Herança

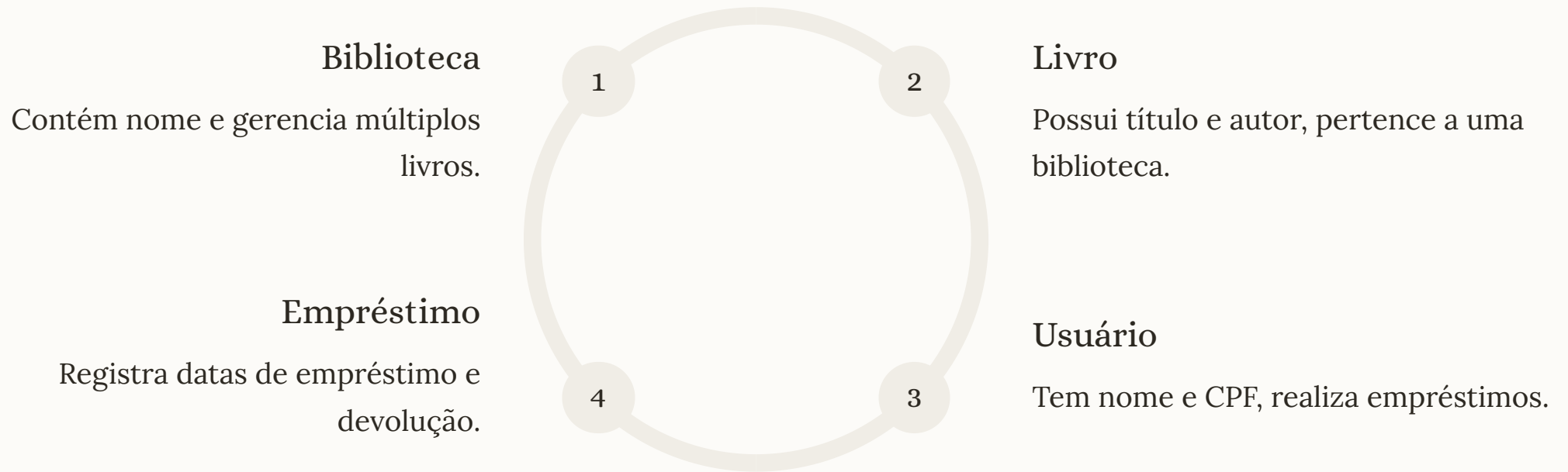
Professor --|> Funcionario: "é-um" (generalização/especialização)



Passos para Criar um Diagrama de Classes



Exemplo Prático - Sistema de Biblioteca



Boas Práticas

1 Simplicidade

Mantenha o diagrama simples e claro, focando no essencial.

2 Nomenclatura

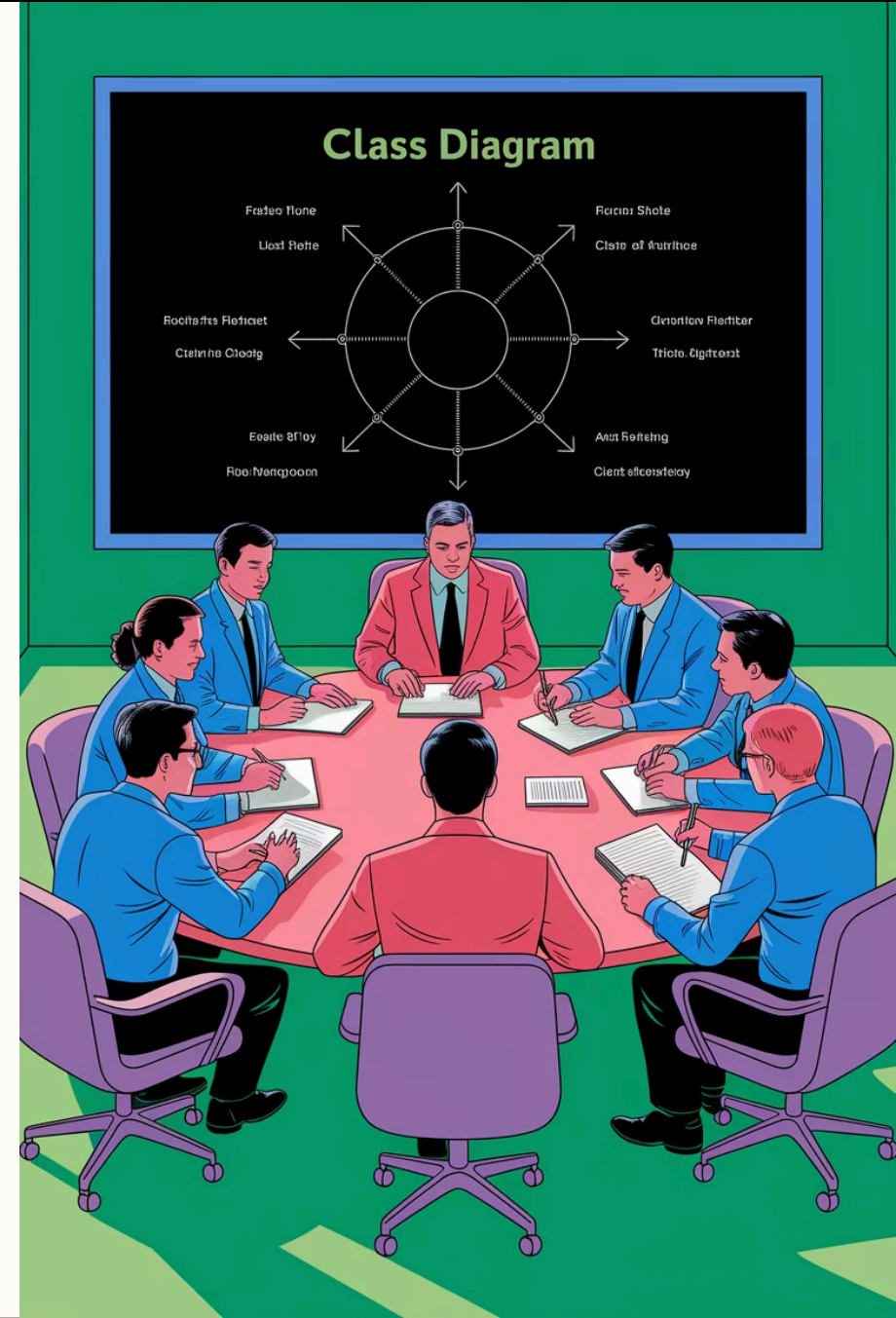
Use nomes significativos para classes, atributos e métodos.

3 Revisão em Equipe

Revise o diagrama com outros membros para garantir consistência.

4 Atualização

Atualize o diagrama conforme o sistema evolui.



Atividade

História de Usuário: Sistema de Gerenciamento de Tarefas

Como um **usuário** , quero poder criar, visualizar e gerenciar minhas tarefas diárias para que eu possa organizar melhor meu tempo e produtividade.

Critérios de Aceitação:

1. O sistema deve permitir que o usuário crie novas tarefas.
2. Cada tarefa deve ter um título, descrição, data de vencimento e status (concluída ou não concluída).
3. O usuário deve poder marcar uma tarefa como concluída.
4. O sistema deve exibir uma lista de todas as tarefas do usuário, separadas por status (concluídas e pendentes).
5. O usuário pode adicionar tags (categorias) às tarefas para facilitar a organização (ex.: "Trabalho", "Pessoal").
6. O sistema deve permitir que o usuário exclua uma tarefa.

Atividade Proposta

Passo 1: Identificar as Classes

- Com base na história de usuário, identifique as principais entidades (classes) do sistema. Perguntas para reflexão:
 - Quais são os principais objetos mencionados na história?
 - Quais informações precisam ser armazenadas para cada objeto?
 - Quais operações (métodos) esses objetos precisam realizar?

Passo 2: Definir Atributos e Métodos

- Para cada classe identificada, liste os atributos e métodos relevantes.
- Considere os critérios de aceitação para determinar os métodos necessários.

Passo 3: Estabelecer Relacionamentos

- Identifique os relacionamentos entre as classes:
 - Associação: Como as classes se conectam?
 - Agregação/Composição: Alguma classe é parte de outra?
 - Herança: Há classes que podem herdar comportamentos de outras?
 - Dependência: Alguma classe depende temporariamente de outra?

Passo 4: Criar o Diagrama de Classes

- Use uma ferramenta online.
- Inclua:
 - Nome das classes.
 - Atributos e métodos.
 - Relacionamentos e multiplicidade.

